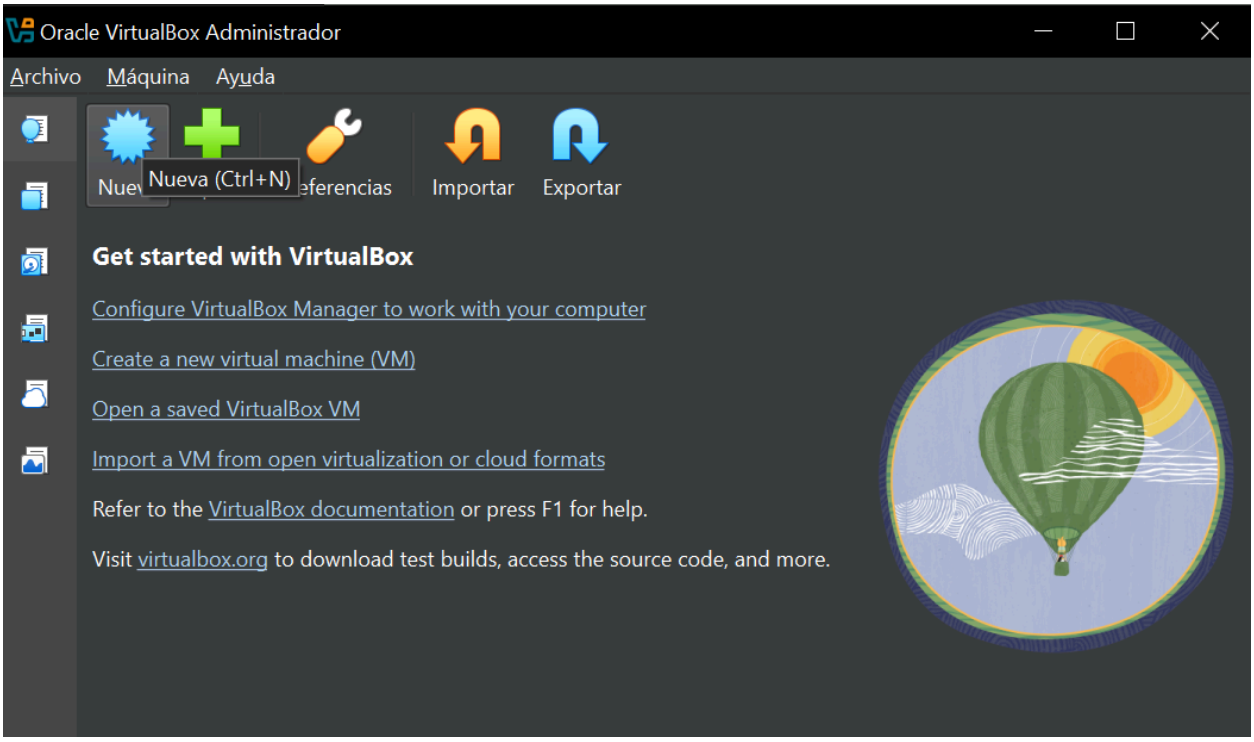
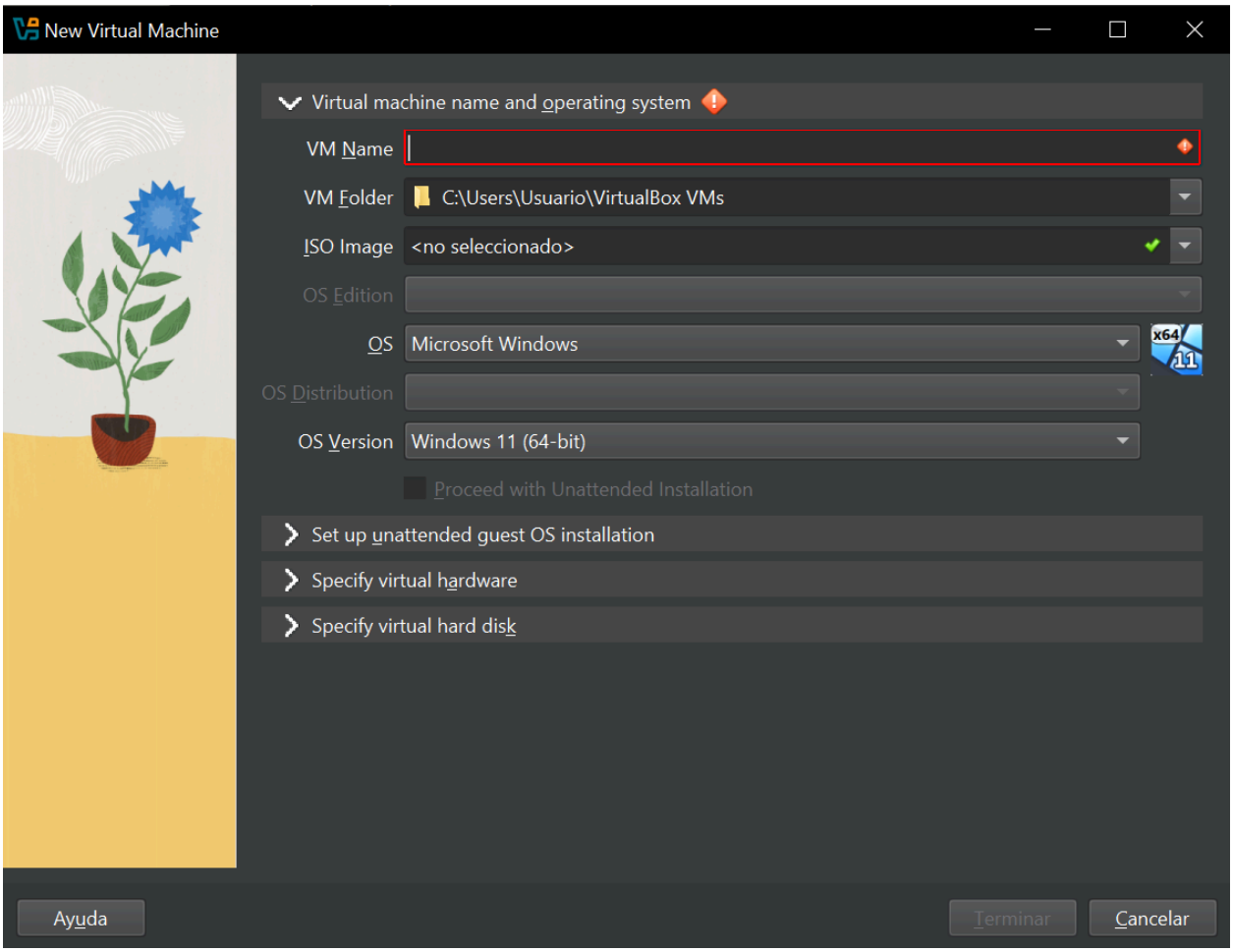
LABORATORIO #1 - Levantamiento de un servidor Linux

ETAPA 1: Crear la máquina virtual en VirtualBox

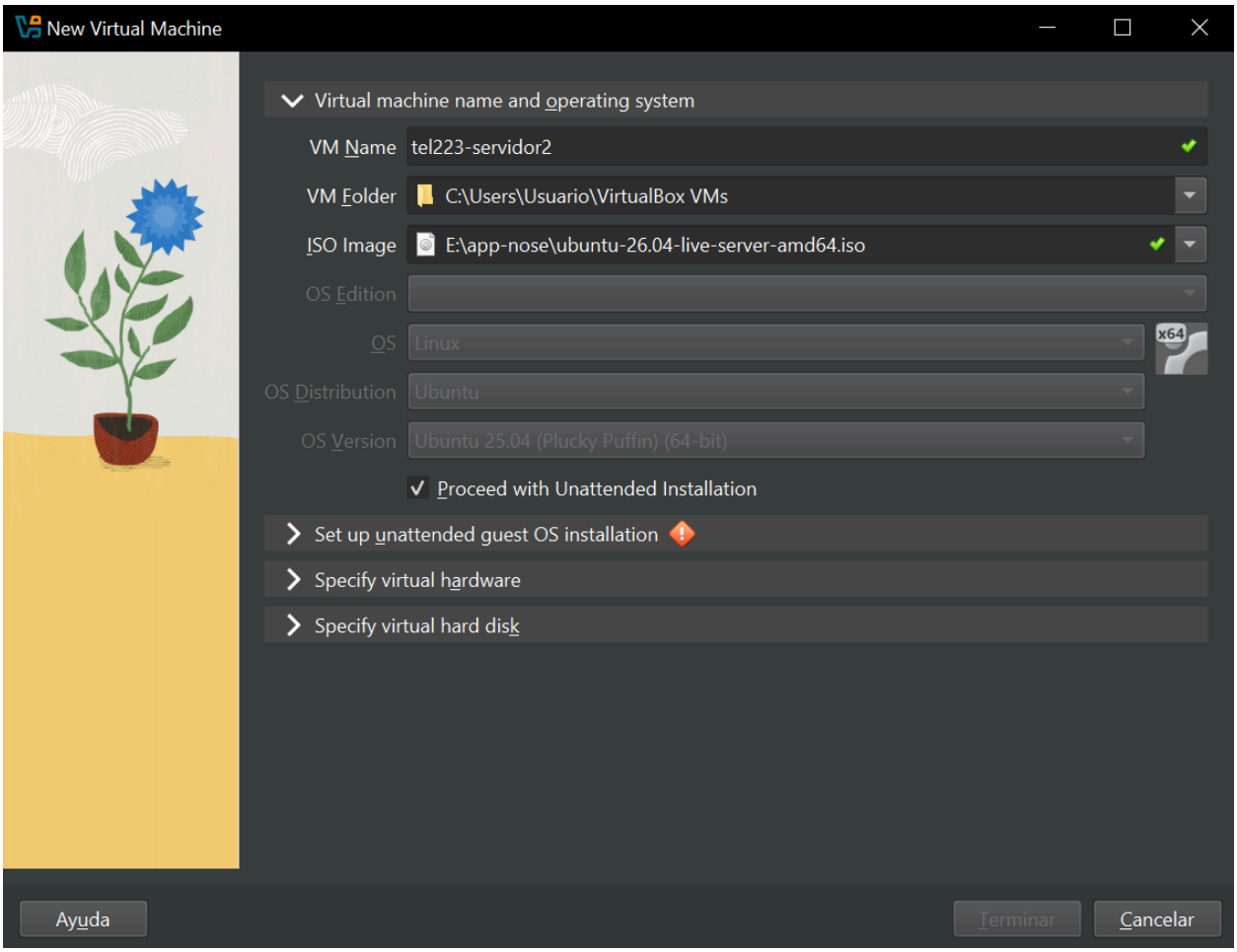
1. **Abrir la aplicación:** Haz doble clic en el icono de VirtualBox en tu escritorio o menú de inicio.
2. **Iniciar la creación:** En la barra superior de VirtualBox, haz clic en el botón **Nueva**.





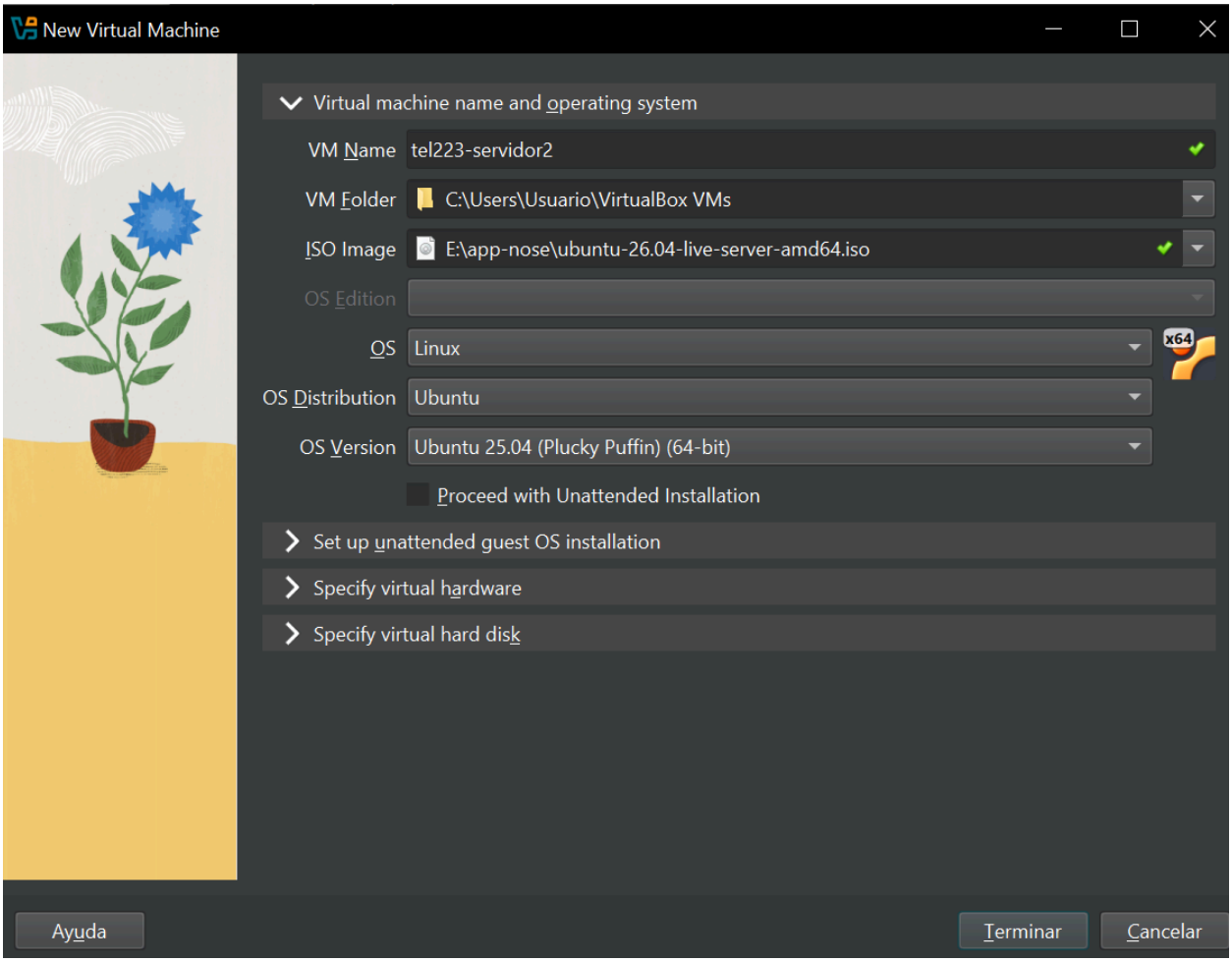
3. **Configuración inicial de la máquina:**

• **Nombre:** Escribe `tel223-servidor`.
• **Carpeta:** Déjala en la ruta por defecto.
• **Imagen ISO:** Haz clic en el menú desplegable, selecciona *Otro...* y busca el archivo de **2,71 GB** ubicado en `E:\app-nose\ubuntu-26.04-live-server-amd64`.
• **Tipo:** Selecciona `Linux`.
• **Versión:** Selecciona `Ubuntu (64-bit)`.
• Casilla obligatoria: Marca la casilla "**Omitir instalación desatendida**" (*Skip Unattended Installation*). Si la dejas desmarcada, no se mostrarán las pantallas interactivas de configuración.
• Presiona **Siguiente**.

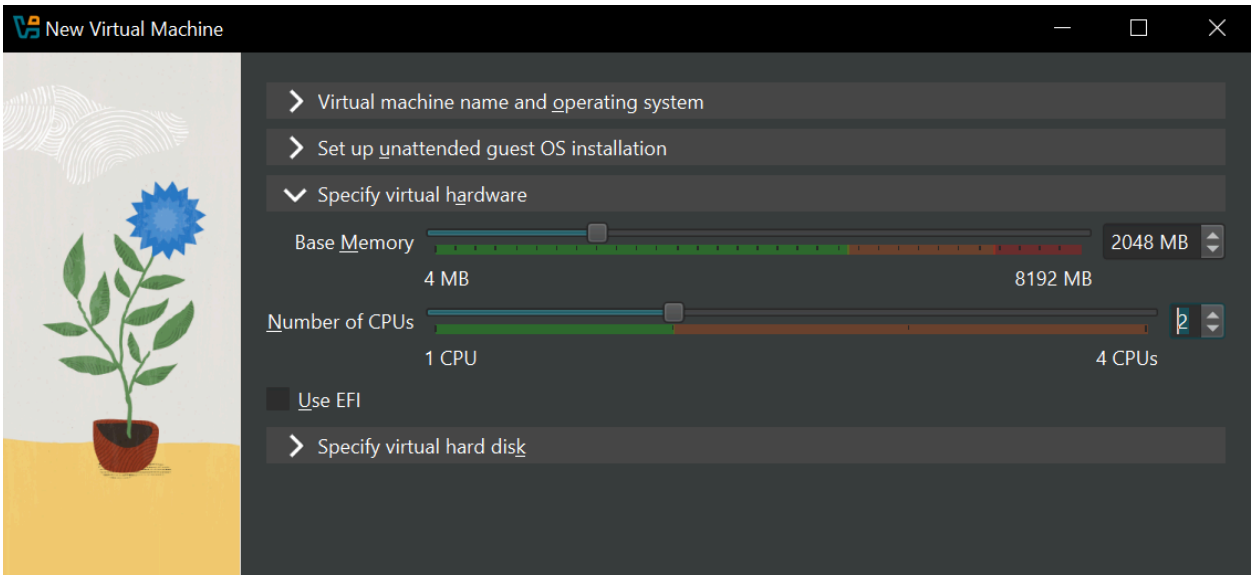


Pasos a seguir en VirtualBox (Etapa 1)

- **Corregir la pantalla actual:** Desmarca la casilla `Proceed with Unattended Installation`.

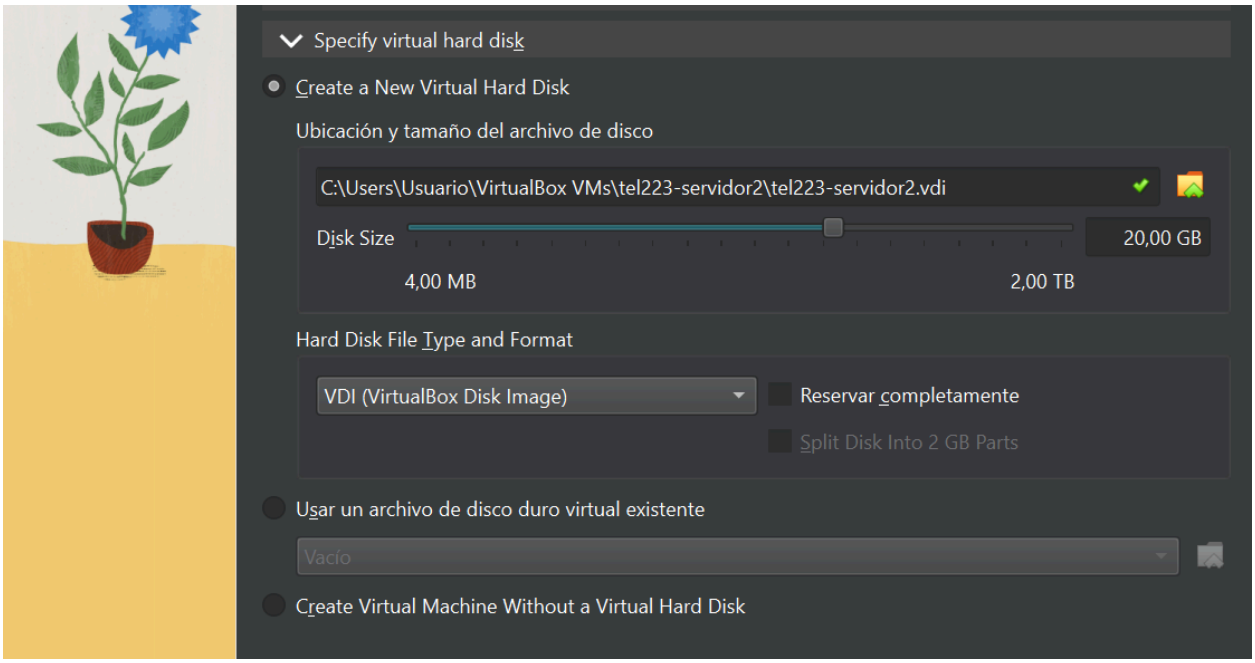


Configurar Hardware (`Specify virtual hardware`): Despliega esa sección y asigna `2048 MB` de memoria RAM y `2` procesadores.

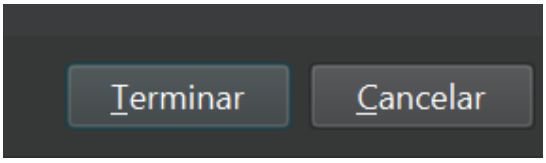


Configurar Disco (`Specify virtual hard disk`): Despliega esa sección y deja el tamaño en `20 GB` con reserva dinámica.

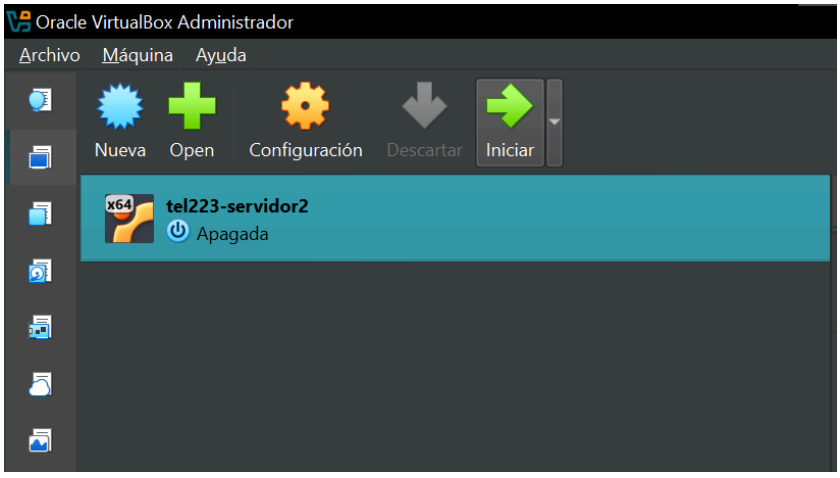
`Specify virtual hard disk` **(NO es totalmente correcta):** El tamaño de **20,00 GB** es correcto, pero debes **DESMARCAR** la casilla `Reservar completamente`. La guía requiere la opción de reserva dinámica para que la máquina ocupe solo unos 4 GB iniciales en tu disco real en lugar de ocupar 20 GB de golpe. `Set up unattended guest OS installation` **(SÍ es correcto):** Es completamente correcto que aparezca deshabilitada y en tono gris. Esto confirma que omitiste la instalación desatendida, permitiendo que se muestren las ventanas interactivas del instalador.



Crear la VM: Haz clic en el botón **Terminar** abajo a la derecha.

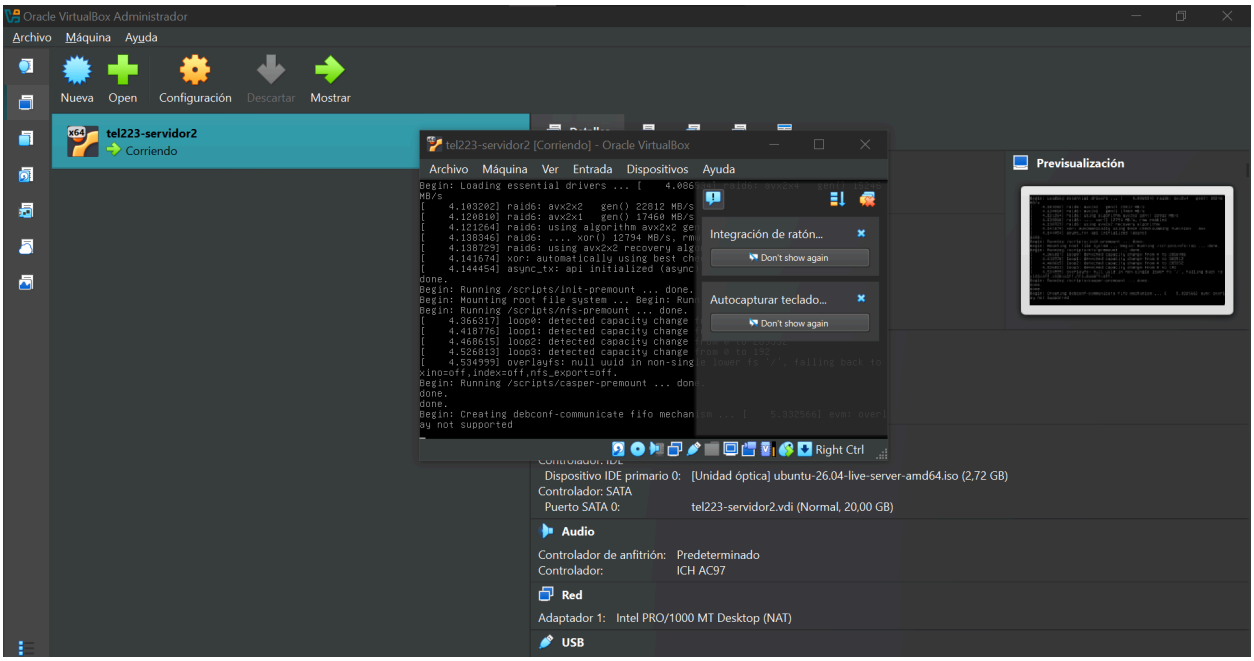


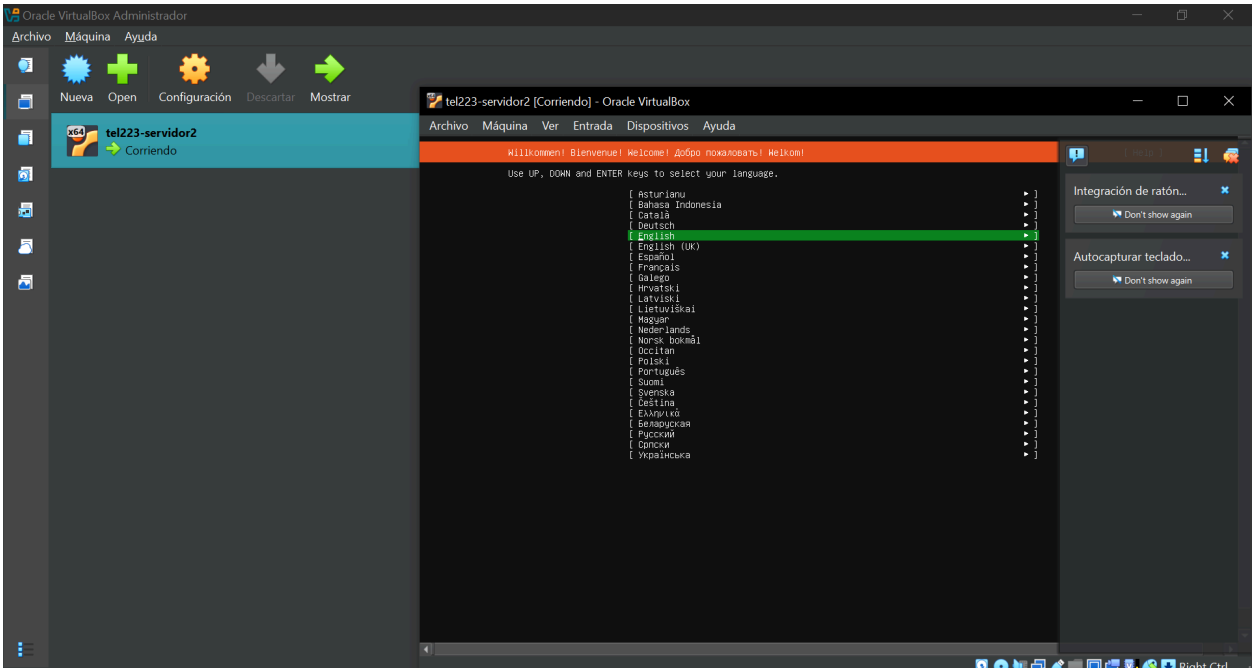
Encender el servidor: Selecciona la máquina en VirtualBox y haz clic en el botón verde **Iniciar**.



[Dar Enter a Try to Install]

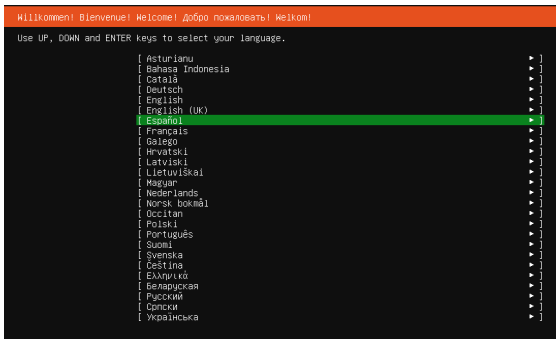
[esperar...]





Completar las pantallas del instalador en este orden exacto:

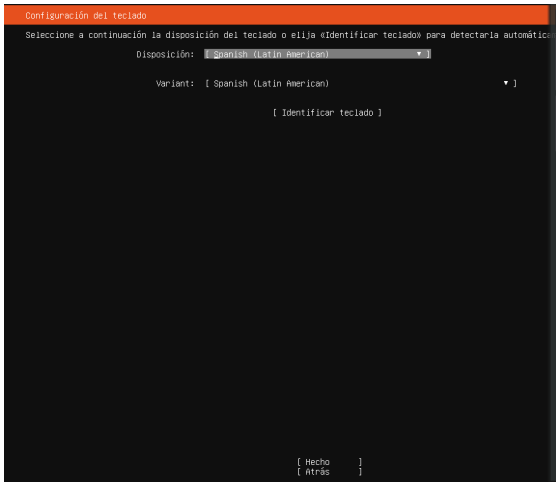
- **Idioma:** Selecciona *English* o *Español*.



• ~~**Actualización del instalador:** Selecciona **Continuar sin actualizar**.~~

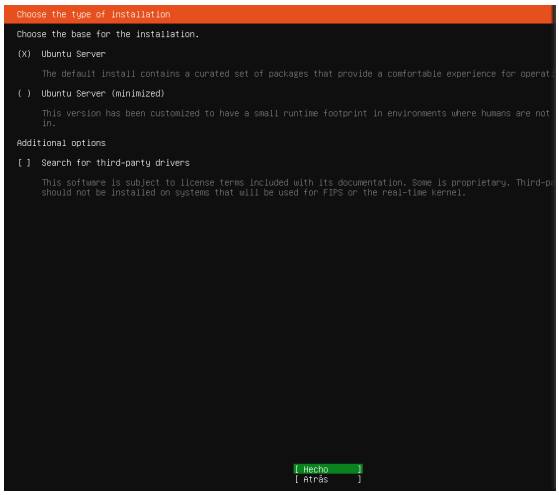
> *Es completamente normal que no te aparezca esa pantalla y puedes continuar con el laboratorio sin ningún problema. La opción de actualización del instalador solo se muestra si la máquina virtual establece conexión a internet en ese segundo preciso; al omitirse, el sistema continúa directamente utilizando el instalador base del disco, logrando exactamente el mismo objetivo que pide la guía (trabajar sin actualizar).*

• **Distribución del teclado:** Configura *Layout: Spanish* y *Variant: Spanish (Latin American)*.

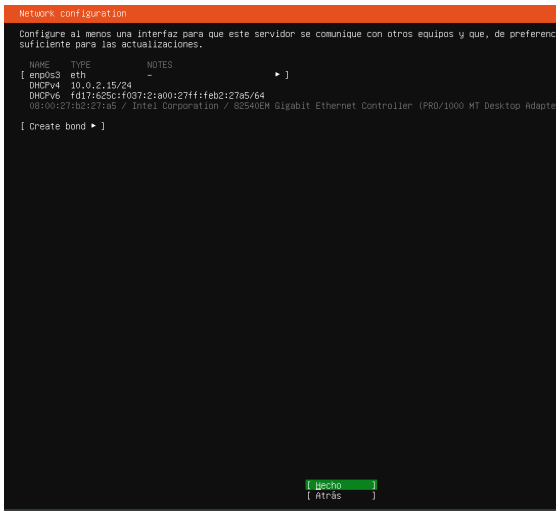


Presionar [Hecho] →

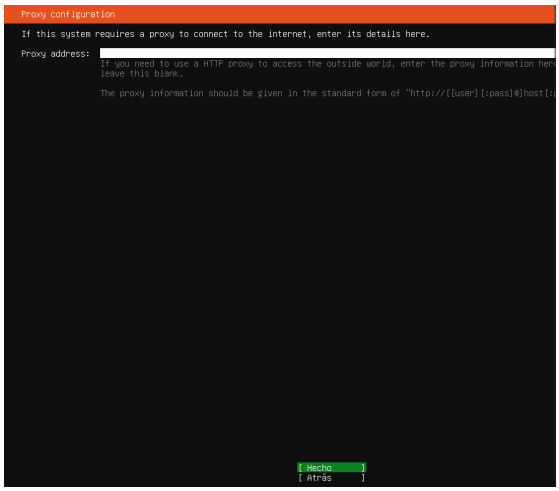
• **Tipo de instalación:** Selecciona **Ubuntu Server** (opción normal).



- **Conexión de red:** Deja la IP asignada por VirtualBox y continúa.

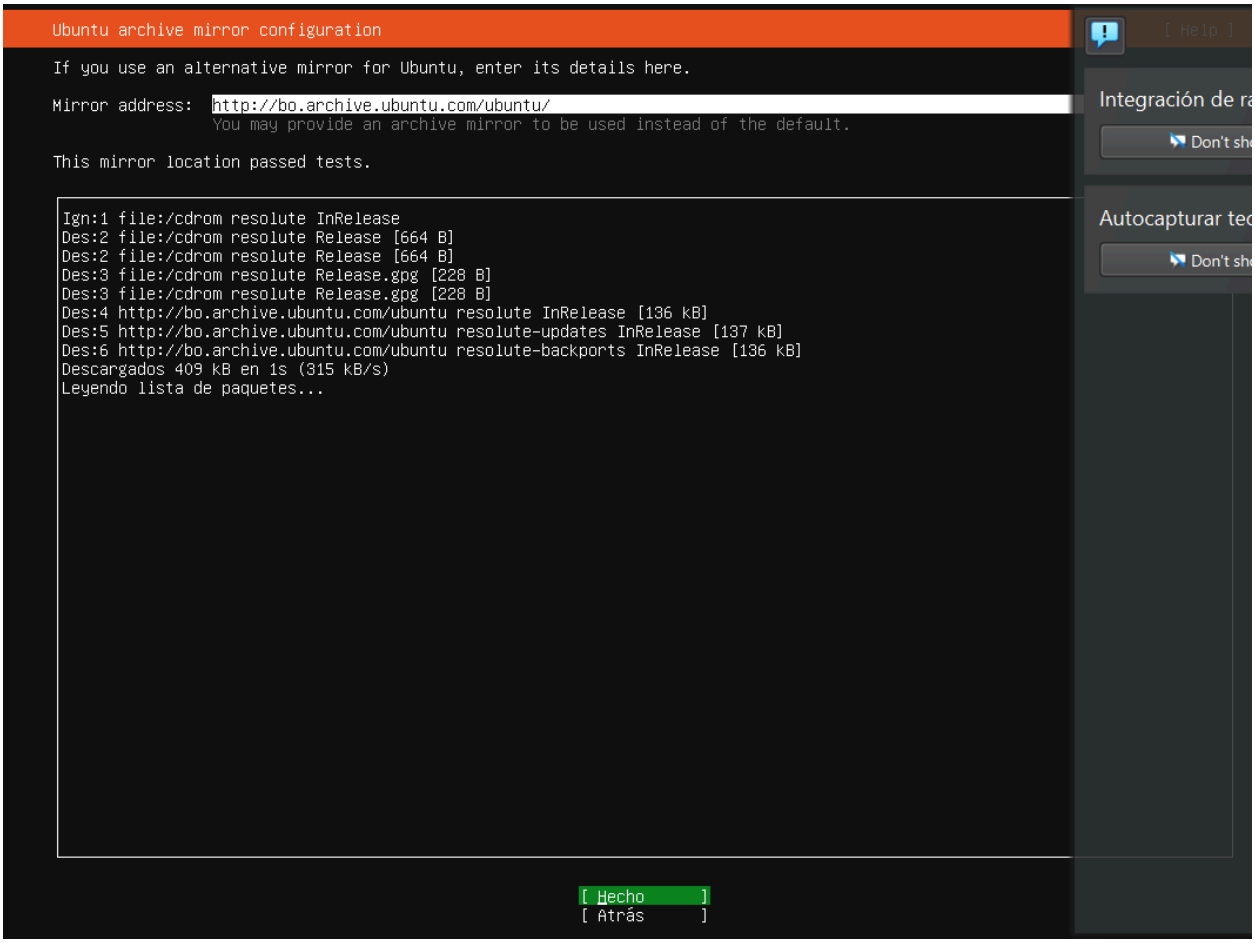


- **Proxy:** Déjalo vacío y presiona continuar.



Réplica de paquetes (Mirror):

• Deja la dirección URL que aparece en pantalla y presiona **Hecho**.



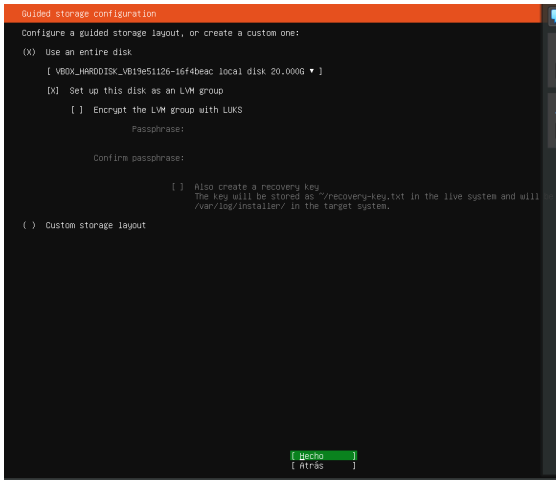
Configuración del disco:

• Elige **Use an entire disk**.

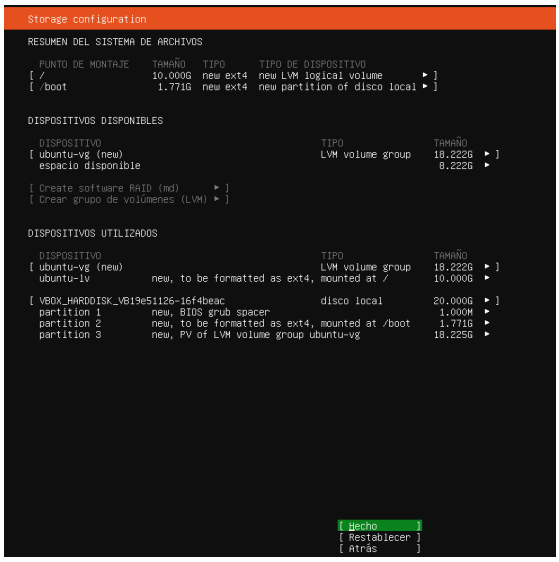
Sí, esa configuración es totalmente correcta y puedes presionar `[ Hecho ]`.

• `Use an entire disk`**:** Es la opción solicitada en la guía para ocupar todo el disco de la máquina virtual.

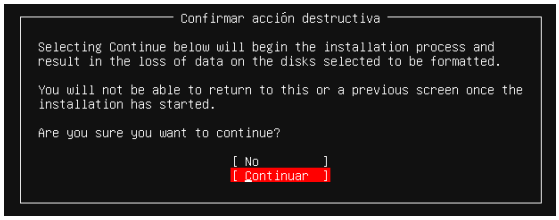
• `Set up this disk as an LVM group`**:** Viene activada por defecto en Ubuntu Server y se mantiene así.



- Presiona **Enter** sobre `[ Hecho ]`
- Te mostrará una pantalla de resumen de almacenamiento; vuelve a presionar `[ Hecho ]`



- Aparecerá un cuadro de diálogo advirtiendo que los datos del disco se borrarán. Selecciona **Continuar** (*Continue*). Recuerda que solo afectará al disco virtual de 20 GB de VirtualBox, tu computadora real está completamente segura.



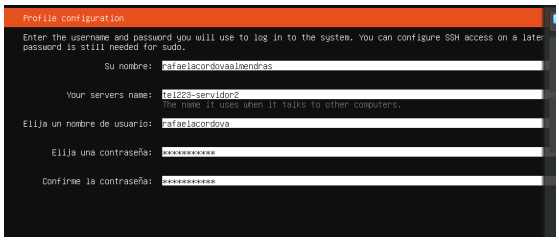
Configuración del perfil:

• **Su nombre:** Tu nombre completo.

• **Nombre del servidor:** `tel223-servidor`.

• **Nombre de usuario:** Tu usuario en minúsculas (sin espacios ni tildes).

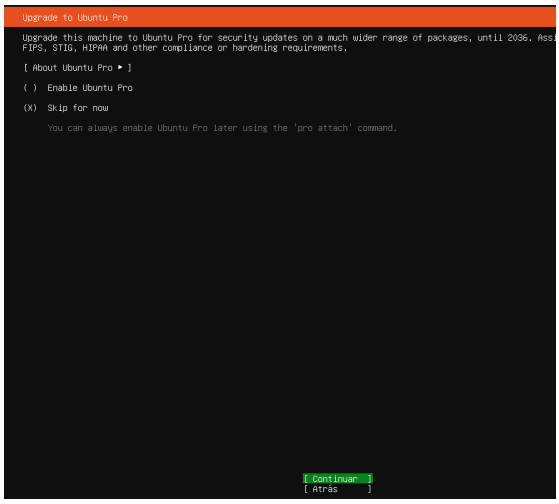
• **Contraseña:** Una clave que recuerdes (anótala, no se puede recuperar).



Clave: toycondiana

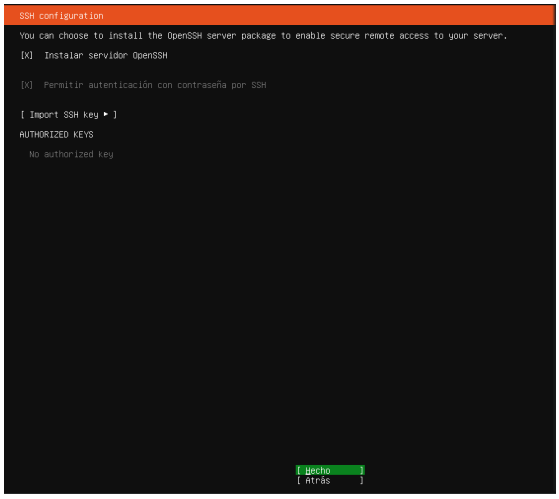
Ubuntu Pro:

• Selecciona **Skip for now** y presiona **Hecho**.



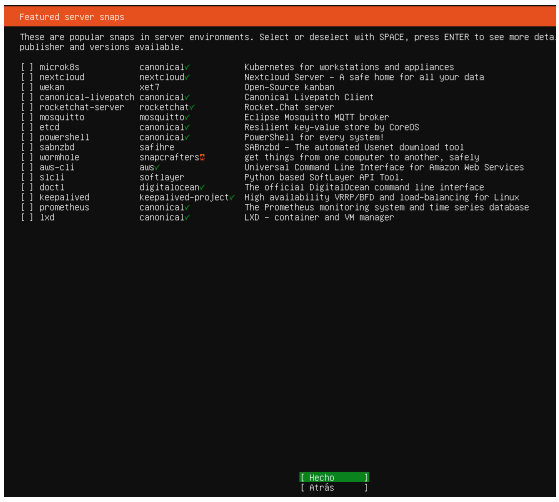
SSH Setup (Paso crucial):

• Usa la **barra espaciadora** para marcar la casilla `Install OpenSSH server` (debe quedar marcada con una `[X]`) y presiona **Hecho**.



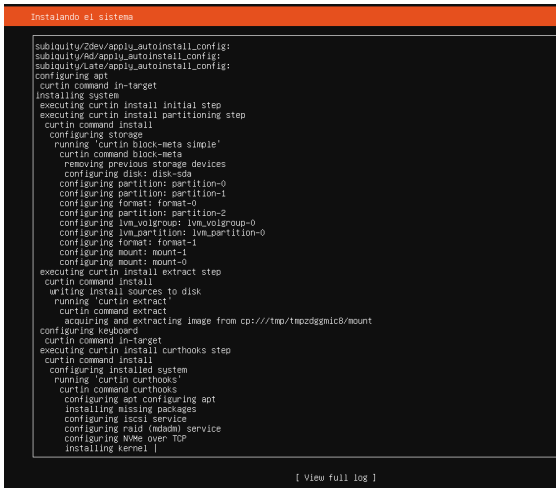
Featured server snaps:

• No marques ninguna opción de la lista; navega hacia abajo hasta **Hecho** y presiona Enter.

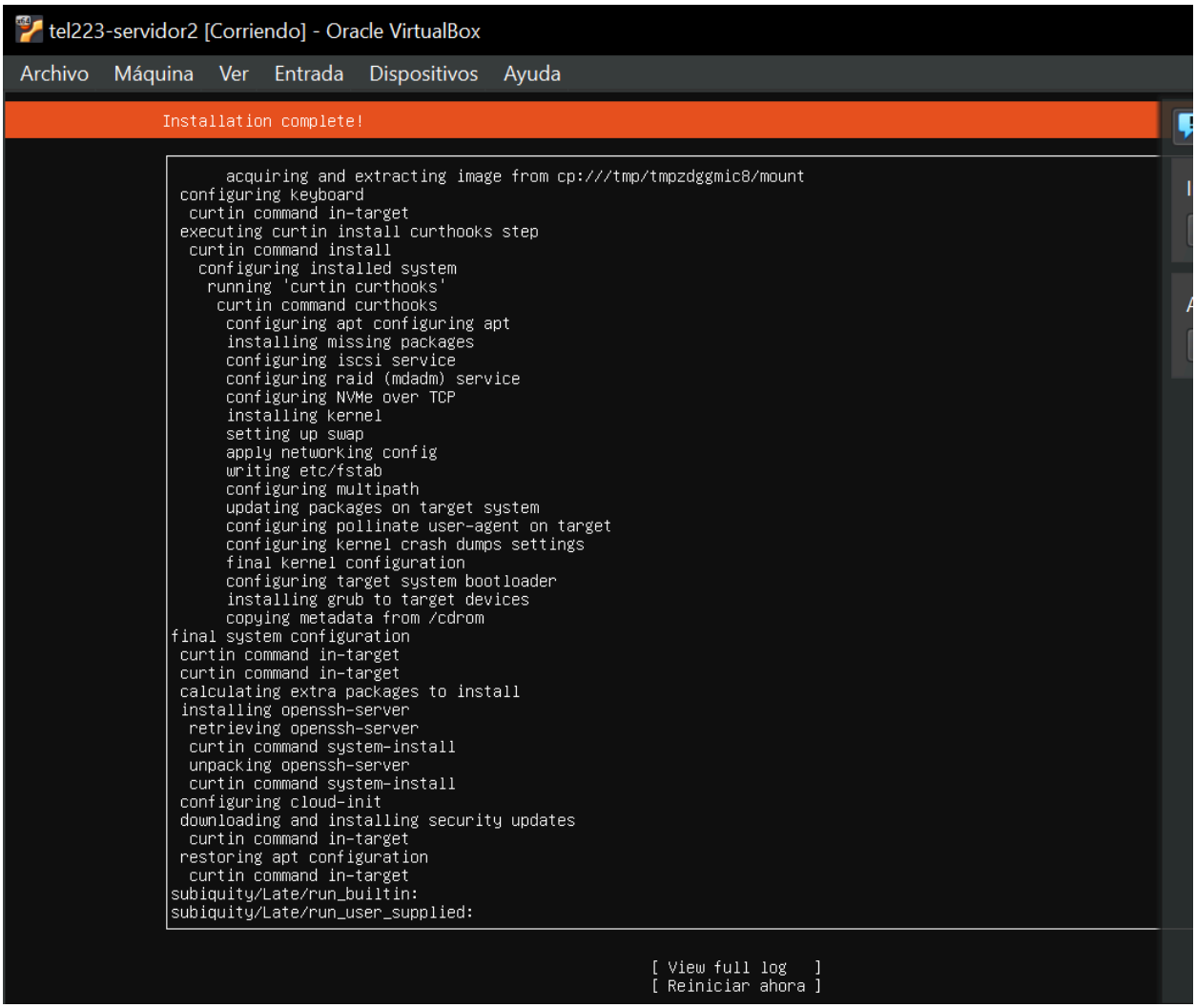


Finalización e inicio:

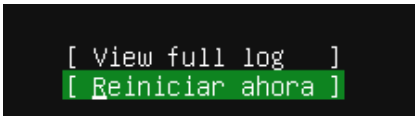
- La instalación comenzará automáticamente.

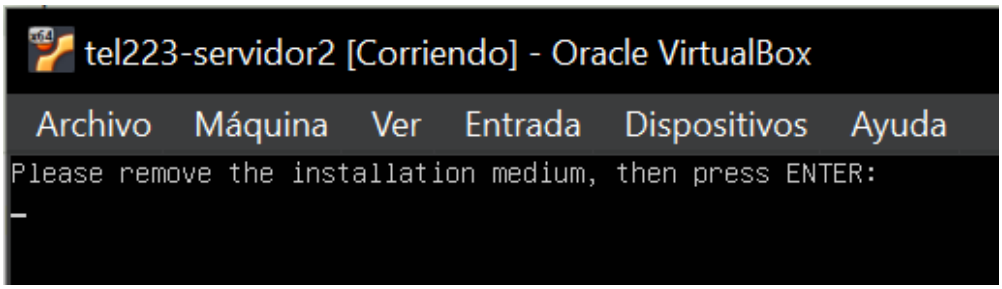


Cuando termine y veas el botón **Reboot Now**, presiónelo. Si la pantalla se detiene pidiendo retirar el medio de instalación, presiona **Enter**.



Sí, está perfectamente bien. El botón te apareció como `[ Reiniciar ahora ]` (abajo a la derecha en la pantalla) porque seleccionaste el idioma español. Es exactamente la misma opción que indica la guía como *Reboot Now*.

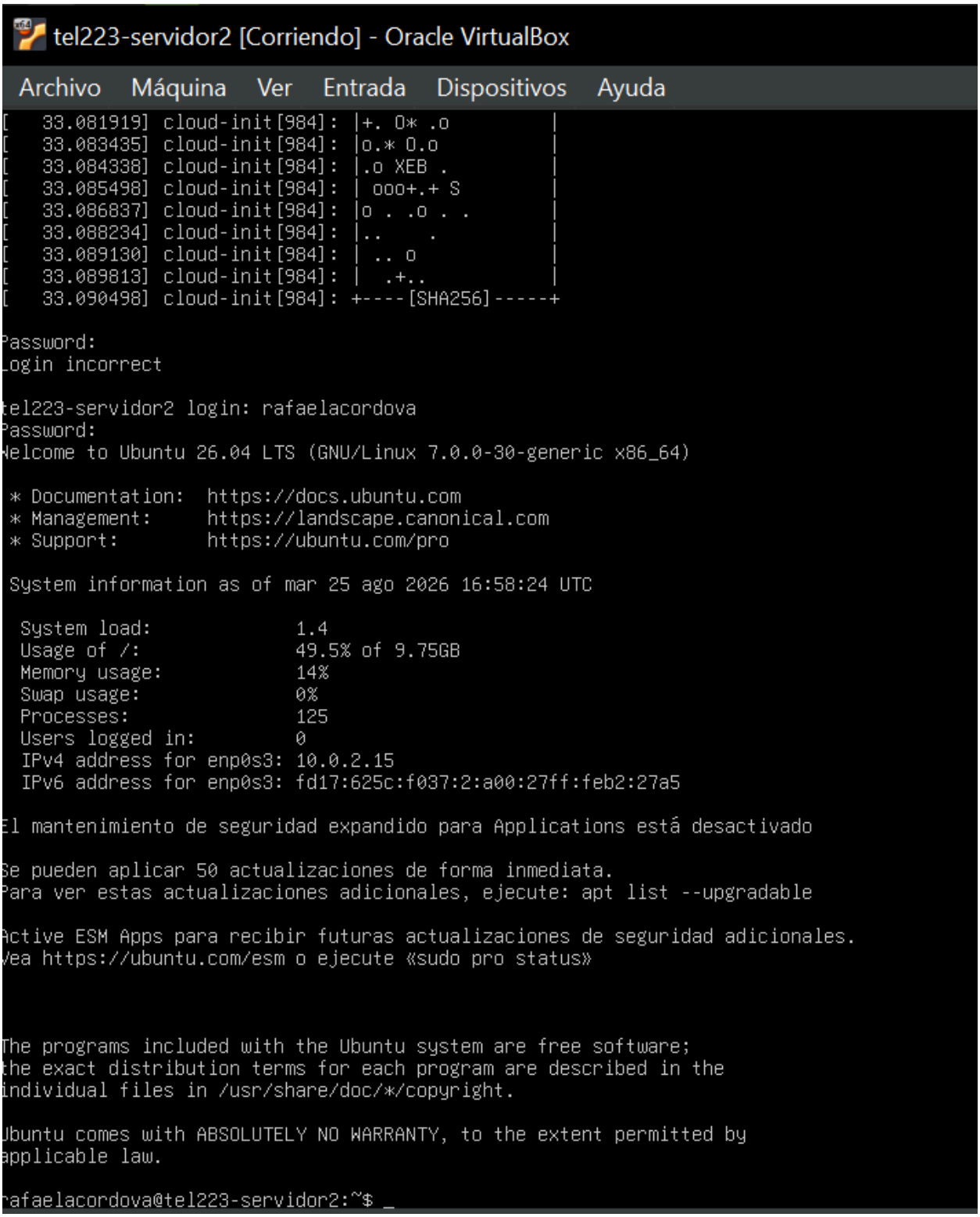




Presiona la tecla **Enter** inmediatamente en tu teclado.

Carga del sistema: La máquina se reiniciará por completo hasta detenerse en la pantalla de inicio de sesión (**PUNTO DE CONTROL 1**).

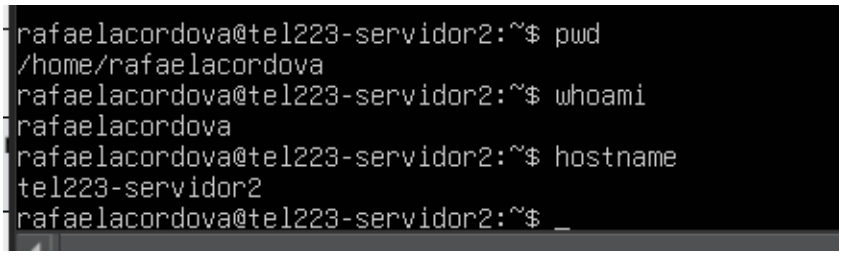
Inicio de sesión (Paso 15): Cuando la pantalla muestre `tel223-servidor2 login:`, escribe tu nombre de usuario en minúsculas y presiona **Enter**. Escribe tu contraseña (no aparecerán asteriscos ni letras en pantalla mientras la escribes) y presiona **Enter**.

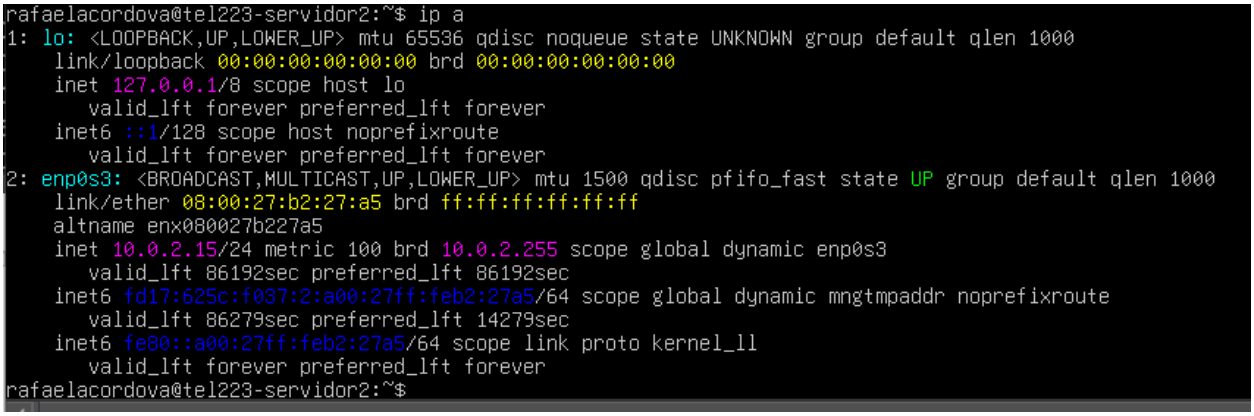


Comandos de la ETAPA 3 (Pasos 16 y 17): Una vez que ingreses y veas el cursor de la terminal, ejecuta estos cuatro comandos uno por uno para completar los espacios de tu

guía:

- `pwd` (muestra la ***ruta de tu directorio*** personal).
- `whoami` (muestra tu nombre de ***usuario***).
- `hostname` (muestra el nombre del ***servidor***).
- `ip a` (busca la línea con la palabra `inet` para ver la dirección IP de tu servidor).





Para completar los datos de la **ETAPA 3** en tu hoja impreso/guía, la dirección IP que aparece en la imagen es `10.0.2.15` (en la sección `2: enp0s3` junto a la palabra `inet`).

A continuación tienes los comandos y pasos exactos que debes escribir en la terminal para continuar y terminar el laboratorio:

ETAPA 4: La jerarquía de archivos

1. Inspecciona la raíz del sistema:
   `ls /`



Es **totalmente correcto**: el resultado muestra la estructura estándar de directorios en la raíz (`/`) de Linux, por lo que puedes continuar con `cd /etc` con total seguridad.

Directorio	Qué contiene	Por qué le importa
/etc	Configuración del sistema y de los servicios	Aquí vive la configuración de todo servicio
/var	Datos que cambian: registros, colas, contenido	En /var/log están los registros
/home	El directorio personal de cada usuario	Aquí está su espacio de trabajo
/usr	Programas y bibliotecas	Aquí viven los comandos que ejecuta

2. Ingresa a la carpeta de configuración y visualiza los primeros elementos:
   `cd /etc`



`ls -l | head`



El comando `ls -l | head` combina dos herramientas mediante una "tubería" (`|`) para inspeccionar el directorio `/etc` de manera ordenada sin saturar la pantalla.

Explicación teórica del comando

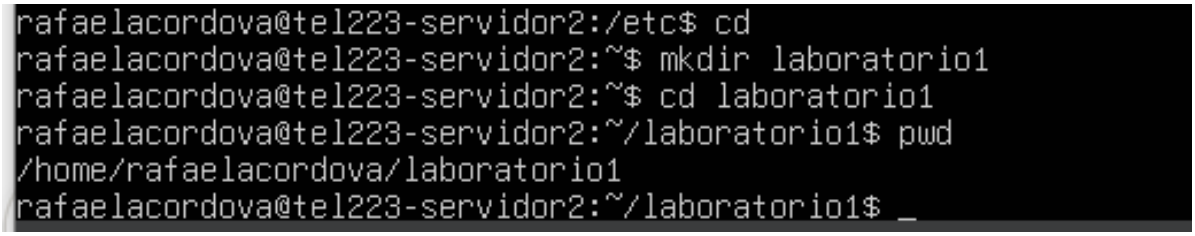
- `ls -l` **:** Muestra la lista detallada de archivos y carpetas con sus propiedades (permisos, propietario, tamaño y fecha).
- `|` **(Tubería / Pipe):** Conecta comandos entre sí. Toma el resultado producido por la izquierda (`ls -l`) y se lo envía directamente como entrada al comando de la derecha (`head`).
- `head` **:** Muestra únicamente las primeras 10 líneas de la lista. Como en `/etc` existen cientos de archivos de configuración, la tubería con `head` evita que la terminal se llene de texto rápido e ilegible.

Análisis de los resultados en la imagen

- `total 920` **:** Indica la suma de bloques de almacenamiento utilizados por la estructura de la carpeta `/etc` en el disco.
- **Tipo de elemento (** `d` **vs):**

    - La `d` al inicio de `drwxr-xr-x` en elementos como `ModemManager`, `PackageKit` o `X11` confirma que son **carpetas** (directorios).
    - El guion inicial en `rw-r--r--` sobre `adduser.conf` indica que es un **archivo de texto regular**.
- `root root` **(Propietario y Grupo):** Todos los archivos de esta carpeta pertenecen al usuario `root` (el administrador del sistema). Esto garantiza que los usuarios normales no puedan modificar las configuraciones del sistema operativo.
- `4027` / `4096`**:** Muestra el tamaño real ocupado en bytes. Por ejemplo, `adduser.conf` pesa 4027 bytes.
- **Nombres de archivos y carpetas notables:**
    - `adduser.conf`**:** Archivo que contiene las reglas globales para cuando creas usuarios nuevos (como los límites de UID o directorios por defecto).
    - `apparmor` / `apparmor.d`**:** Carpetas que contienen los perfiles del sistema de seguridad que protege al núcleo de Linux.
    - `X11`**:** Carpeta reservada para las configuraciones del entorno gráfico de la pantalla.

3. Regresa a tu directorio personal y crea tu espacio de trabajo:
   `cd`
   `mkdir laboratorio1`
   `cd laboratorio1`
   `pwd`



ETAPA 5: Usuarios, grupos y permisos

1. Consulta tu identidad y grupos:

   `id`

```
rafaelacordova@tel223-servidor2:~/laboratorio1$ id
uid=1000(rafaelacordova) gid=1000(rafaelacordova) groups=1000(rafaelacordova),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),101(lxd)
```

`groups`

```
rafaelacordova@tel223-servidor2:~/laboratorio1$ groups
rafaelacordova adm cdrom sudo dip plugdev users lxd
```

Los comandos `id` y `groups` muestran la credencial digital de tu cuenta y ***confirman*** que posees ***privilegios de administrador para gestionar el servidor.*** En Linux, la ***seguridad*** se gestiona asignando un ***número de identificación*** a cada usuario y organizándolos en grupos para otorgar o restringir permisos.

Imagen 1: Comando `id`

Muestra la ficha técnica completa de tu usuario con sus identificadores numéricos y de texto:

- `uid=1000(rafaelacordova)`**:** *User ID*. Es el número único que el sistema operativo asigna a tu cuenta. En Ubuntu, los ***usuarios normales*** creados tras la instalación comienzan a partir del número 1000.
- `gid=1000(rafaelacordova)`**:** *Group ID*. Representa tu grupo primario predeterminado.
- `groups=...`**:** Enumera todos los grupos secundarios a los que perteneces junto con su ID numérico:
  - `27(sudo)`**:** El grupo más importante. Te otorga la capacidad de ejecutar acciones administrativas utilizando el comando `sudo`.
  - `4(adm)`**:** Te permite leer los archivos de registro y eventos del sistema (*logs*).
  - `24(cdrom)`**,** `30(dip)`**,** `46(plugdev)`**:** Otorgan acceso a dispositivos de almacenamiento óptico, herramientas de marcado/red y dispositivos externos removibles.
  - `100(users)`**,** `101(lxd)`**:** Grupos estándar para usuarios generales y gestión de contenedores virtuales.

Imagen 2: Comando `groups`
Ofrece un resumen simplificado de la lista de grupos, omitiendo los códigos numéricos para facilitar su lectura rápida.

- Muestra directamente los nombres de las agrupaciones a las que está vinculado tu usuario: `rafaelacordova adm cdrom sudo dip plugdev users lxd`.
- La presencia de la palabra `sudo` en esta lista confirma formalmente que eres uno de los administradores principales del servidor.

MÁS PREGUNTAS DE LOS OTROS GRUPOS:

Explicado sin tecnicismos: estos grupos son simplemente "pases de acceso" que te otorgan permiso para usar partes específicas de la computadora sin tener que escribir tu clave de administrador cada vez.

- `cdrom` **(Lectora de discos):** Te da permiso para leer o grabar discos CD/DVD si tu computadora tiene lectora física.
- `plugdev` **(Memorias USB y discos externos):** Te permite conectar un pendrive o disco duro por USB y abrir sus carpetas directamente como un usuario normal.
- `dip` **(Conexiones por módem):** Es un permiso antiguo que quedó guardado en Linux. Servía para conectar la computadora a internet haciendo una llamada telefónica mediante un módem.
- `users` **(Usuarios comunes):** Es el grupo estándar del sistema. Es el club general al que pertenece cualquier cuenta normal creada en la máquina.
- `lxd` **(Mini máquinas virtuales):** Te da autorización para crear y manejar "contenedores", que son como pequeños servidores virtuales independientes ejecutándose dentro de tu mismo Ubuntu.

1. Crea un archivo y verifica a quién pertenece:
   `touch notas.txt`

`ls -l notas.txt`

```
rafaelacordova@tel223-servidor2:~/laboratorio1$ touch notas.txt
rafaelacordova@tel223-servidor2:~/laboratorio1$ ls -l notas.txt
-rw-rw-r-- 1 rafaelacordova rafaelacordova 0 Aug 25 17:46 notas.txt
rafaelacordova@tel223-servidor2:~/laboratorio1$
```

🗨 **Explicación teórica y de tu resultado**

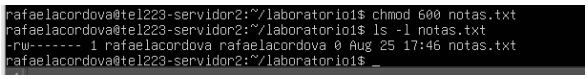
En Linux, cada vez que creas un archivo, el sistema le pega una "etiqueta de identificación" que define tres cosas: **quién es el dueño**, **a qué grupo pertenece** y **qué acciones están permitidas hacer con él** (leer, escribir o ejecutar).

Analizando la línea que te devolvió el comando:
`-rw-rw-r-- 1 rafaelacordova rafaelacordova 0 Aug 25 17:46 notas.txt`

- `-rw-rw-r--` **(Los Permisos de acceso):**
  - El primer guion `-`: Indica que es un **archivo normal** (si fuera una carpeta/directorio, llevaría una `d`).
  - Primer `rw-`: **Tus permisos (Propietario).** Puedes leer (`r` = *read*) y escribir/modificar (`w` = *write*).
  - Segundo `rw-`: **Permisos de tu grupo.** Los usuarios que pertenezcan a tu grupo principal también pueden leer y escribir.
  - Tercer `r--`: **Permisos de Todos los demás.** Cualquier otro usuario del servidor únicamente puede leer/ver el archivo, pero no modificarlo.
- `1` **(Número de enlaces):** Indica cuántas referencias físicas apuntan a este archivo.
- `rafaelacordova rafaelacordova` **(Dueño y Grupo):** El primer nombre indica que **tú eres el propietario** (por haberlo creado) y el segundo nombre es el **grupo propietario**.
- `0` **(Tamaño en disco):** Indica que pesa **0 bytes**. Esto se debe a que el comando `touch` simplemente crea "una hoja en blanco" vacía.
- `Aug 25 17:46` **(Estampa de tiempo):** La fecha y hora exacta en la que se creó o modificó el archivo por última vez.
- `notas.txt` **(Nombre):** El nombre del archivo que acabas de consultar.

1. Modifica los permisos para hacer el archivo privado y comprueba el efecto:
   `chmod 600 notas.txt`

   `ls -l notas.txt`



El resultado es **100 % correcto**: acabas de convertir el archivo `notas.txt` en un documento estrictamente privado.

Explicación teórica (El sistema de números `600`)

El comando `chmod` (*Change Mode*) altera las reglas de acceso de un archivo usando un código numérico de 3 dígitos. En Linux, los permisos se otorgan mediante puntos:

- **4 puntos:** Permiso de **Leer** (*Read*)
- **2 puntos:** Permiso de **Escribir/Modificar** (*Write*)
- **1 punto:** Permiso de **Ejecutar** (*Execute*)

Al escribir `600`, le asignas una combinación a tres destinatarios distintos:

- **Primer dígito `6` (Tú / Propietario):** Es la suma de **4 + 2**. Significa que tú tienes permiso de **leer** y **modificar** el archivo.
- **Segundo dígito `0` (Tu grupo): 0 puntos**. Los miembros de tu grupo no pueden ni leerlo ni editarlo.
- **Tercer dígito `0` (Otros usuarios): 0 puntos**. Cualquier otra persona en la computadora tiene el acceso totalmente bloqueado.**En resumen:** El código `600` equivale a guardar el archivo en una caja fuerte de la que solo tú tienes la clave.

Análisis de tu resultado en la imagen

Al ejecutar `ls -l notas.txt`, la pantalla te devuelve:

```
-rw------- 1 rafaelacordova rafaelacordova 0 Aug 25 17:46 notas.txt
```

- `-rw-------` **(Nuevos permisos):**
  - El primer guion `-`: Confirma que es un archivo normal.
  - La sección `rw-`: Confirma que tú (`rafaelacordova`) tienes permisos de lectura (`r`) y escritura (`w`).
  - La sección `------`: Los 6 guiones seguidos demuestran que el grupo y los demás usuarios se quedaron con **cero permisos**.

1. Restaura el permiso de lectura general:

   ```
   chmod 644 notas.txt
   ```

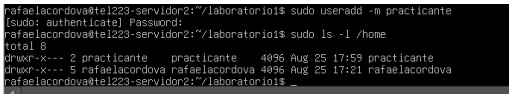
   ```
   ls -l notas.txt
   ```

```
rafaelacordova@tel223-servidor2:~/laboratorio1$ chmod 644 notas.txt
rafaelacordova@tel223-servidor2:~/laboratorio1$ ls -l notas.txt
-rw-r--r-- 1 rafaelacordova rafaelacordova 0 Aug 25 17:46 notas.txt
```

2. Crea un nuevo usuario administrado (te pedirá ingresar tu contraseña):
   `sudo useradd -m practicante`

   `sudo ls -l /home`

   (Ingresa tu contraseña si te la pide; al final verás las carpetas `rafaelacordova` *y* `practicante` *en* `/home` *)*





El resultado es **100 % exitoso**: creaste correctamente el nuevo usuario llamado `practicante` y el sistema le generó su propia carpeta personal.
Explicación teórica del comando (`sudo useradd -m practicante`)

- `sudo`**:** Es la orden que usas para actuar como administrador ("poner tu firma de autorización").
- `useradd`**:** Es el comando especializado de Linux para dar de alta una nueva cuenta de usuario en el sistema.
- `m`**:** Proviene de *make home*. Obliga a Linux a construir automáticamente la "casa" o carpeta personal del nuevo usuario dentro del directorio `/home`.
- `[sudo: authenticate] Password:`**:** Es el candado de seguridad del sistema solicitando tu contraseña personal para confirmar que tienes permiso de crear usuarios.

Análisis del resultado en la pantalla (`sudo ls -l /home`)
Al listar el contenido de `/home`, la pantalla te muestra dos registros: `drwxr-x--- 2 practicante practicante 4096 Aug 25 17:59 practicantedrwxr-x--- 5 rafaelacordova rafaelacordova 4096 Aug 25 17:21 rafaelacordova`

- `d` **(Al inicio):** La letra `d` en `drwxr-x---` confirma que **es un directorio (carpeta)** y no un archivo suelto.
- `practicante practicante`**:** El primer nombre indica que el **propietario** de la carpeta es el usuario `practicante`, y el segundo indica su **grupo**

exclusivo.

- **Confirmación visual:** La aparición de la carpeta `practicante` al lado de la tuya (`rafaelacordova`) demuestra que el usuario ya existe legalmente en el servidor y posee su propio espacio de almacenamiento. Con esto completas el **PUNTO DE CONTROL 3** de tu guía.

🗨️ En la cadena `drwxr-x---`, los permisos se dividen en tres bloques de 3 caracteres después de la `d` inicial (que indica *directorio/carpeta*):

1. `rwx` → Permisos del Propietario (Dueño).
2. `r-x` → Permisos del Grupo.
3. `--` → Permisos de Otros usuarios.

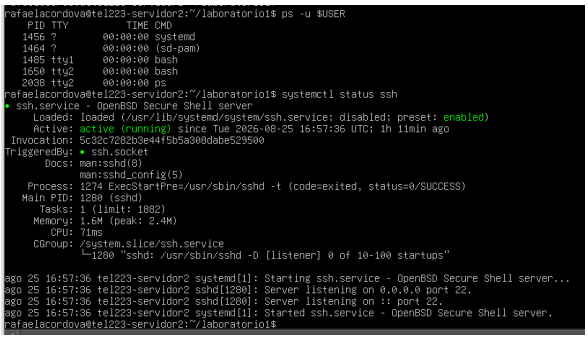
La combinación `r-x---` representa las reglas para el grupo y para los desconocidos:

- `r-x` **(El Grupo):**
  - `r` **(*Read*):** Pueden **ver** qué archivos hay dentro.
  - **(*No Write*):** El guion ocupa el lugar de la `w`. Significa que **no pueden** crear, borrar ni modificar archivos dentro.
  - `x` **(*eXecute*):** Les permite **entrar** a la carpeta (usar el comando `cd`).
- `--` **(Otros usuarios):**
  - Tres guiones juntos significan **cero permisos**. Un usuario ajeno no puede ni ver el contenido, ni escribir, ni entrar a la carpeta.

ETAPA 6: Servicios y fallas provocadas

1. Revisa tus procesos activos y el servicio SSH (si la pantalla se llena, presiona la tecla `q` para salir):
   `ps -u $USER`

   `systemctl status ssh`



🗨️ **Comando 1: `ps -u $USER` (Monitoreo de procesos)**

• **Teoría:** El comando `ps` (*Process Status*) toma una foto de los programas cargados en la memoria RAM. Al agregar `-u $USER`, el sistema filtra la lista para mostrar únicamente las tareas pertenecientes a tu usuario actual (`rafaelacordova`).

• **Análisis de tu resultado:**

◦ `PID` **(Process ID):** Número único de identificación que el sistema asigna a cada tarea en ejecución.

◦ `TTY`**:** La consola o terminal virtual desde la cual se ejecutó el programa (`tty1` o `tty2`).

◦ `CMD`**:** El nombre del programa. En tu pantalla ves `bash` (tu intérprete de comandos) y `ps` (el comando de consulta que acabas de lanzar).

🗨️ **Comando 2: `systemctl status ssh` (Estado de servicios)**

• **Teoría:** En Linux, los programas que trabajan continuamente en segundo plano (como servidores de red) se llaman *servicios*. El comando `systemctl status` consulta a la herramienta administrativa `systemd` para saber si un servicio está encendido, apagado o fallando.

• **Análisis de tu resultado:**

◦ `Active: active (running)` **(en verde):** Confirma que el servicio de conexión remota SSH está encendido y funcionando perfectamente en segundo plano.

◦ `Main PID: 1280 (sshd)`**:** Muestra que el proceso principal de SSH está corriendo en la memoria bajo el identificador 1280.

El **Main PID** (*Main Process Identifier*) es el número de documento de identidad único que el núcleo de Linux le asigna al proceso principal

responsable de mantener un servicio funcionando.
¿Qué es y para qué sirve?
Un servicio puede generar decenas de subprocesos o tareas secundarias, pero `systemd` necesita rastrear al "jefe de orquesta". El Main PID es la cabeza del servicio: si ese proceso específico desaparece o se detiene, el sistema sabe de inmediato que el servicio cayó o fue apagado.
Desglose de partes cuando el servicio está ACTIVO
Ejemplo: `Main PID: 1280 (sshd)`

- `Main PID:`: La etiqueta con la que `systemd` indica que ese es el proceso líder del servicio.
- `1280` **(Número de PID):** El identificador numérico único asignado en la memoria RAM. Este número cambia cada vez que el servicio se reinicia.
- `(sshd)` **(Ejecutable):** El nombre del programa real que corre en la memoria (en este caso, el demonio de SSH).

Desglose de partes cuando el servicio está INACTIVO o DETENIDO
Ejemplo: `Main PID: 1280 (code=exited, status=0/SUCCESS)`

- `1280` **(Último PID):** El número que tenía el proceso antes de cerrar.
- `code=exited` **(Razón de salida):** Indica la forma en que cerró. `exited` significa que el programa completó su orden de salida y se retiró (diferente a `killed`, que indicaría un cierre forzado o colapso por falta de memoria).
- `status=0` **(Código numérico de salida):** En los sistemas UNIX y Linux, el código **0** es el estándar universal para *"operación exitosa sin errores"*.
- `SUCCESS` **(Verificación):** La confirmación legible para el usuario de que el cierre fue limpio y planificado.

◦ `Server listening on 0.0.0.0 port 22` **:** Los registros al final confirman que el servidor está a la espera de conexiones entrantes por el puerto de red 22.
El **Main PID** (*Main Process Identifier*) es el número de documento de identidad único que el núcleo de Linux le asigna al proceso principal responsable de mantener un servicio funcionando.
¿Qué es y para qué sirve?

Un servicio puede generar decenas de subprocesos o tareas secundarias, pero `systemd` necesita rastrear al "jefe de orquesta". El Main PID es la cabeza del servicio: si ese proceso específico desaparece o se detiene, el sistema sabe de inmediato que el servicio cayó o fue apagado.

Desglose de partes cuando el servicio está ACTIVO

Ejemplo: `Main PID: 1280 (sshd)`

- `Main PID:` : La etiqueta con la que `systemd` indica que ese es el proceso líder del servicio.
- `1280` **(Número de PID):** El identificador numérico único asignado en la memoria RAM. Este número cambia cada vez que el servicio se reinicia.
- `(sshd)` **(Ejecutable):** El nombre del programa real que corre en la memoria (en este caso, el demonio de SSH).

Desglose de partes cuando el servicio está INACTIVO o DETENIDO

Ejemplo: `Main PID: 1280 (code=exited, status=0/SUCCESS)`

- `1280` **(Último PID):** El número que tenía el proceso antes de cerrar.
- `code=exited` **(Razón de salida):** Indica la forma en que cerró. `exited` significa que el programa completó su orden de salida y se retiró (diferente a `killed`, que indicaría un cierre forzado o colapso por falta de memoria).
- `status=0` **(Código numérico de salida):** En los sistemas UNIX y Linux, el código **0** es el estándar universal para *"operación exitosa sin errores"*.
- `SUCCESS` **(Verificación):** La confirmación legible para el usuario de que el cierre fue limpio y planificado.

El **Main PID** (*Main Process Identifier*) es el número de documento de identidad único que el núcleo de Linux le asigna al proceso principal responsable de mantener un servicio funcionando.

¿Qué es y para qué sirve?

Un servicio puede generar decenas de subprocesos o tareas secundarias, pero `systemd` necesita rastrear al "jefe de orquesta". El Main PID es la cabeza del servicio: si ese proceso específico desaparece o se detiene, el sistema sabe de inmediato que el servicio cayó o fue apagado.

Desglose de partes cuando el servicio está ACTIVO

Ejemplo: `Main PID: 1280 (sshd)`

- `Main PID:`: La etiqueta con la que `systemd` indica que ese es el proceso líder del servicio.
- `1280` **(Número de PID):** El identificador numérico único asignado en la memoria RAM. Este número cambia cada vez que el servicio se reinicia.
- `(sshd)` **(Ejecutable):** El nombre del programa real que corre en la memoria (en este caso, el demonio de SSH).

Desglose de partes cuando el servicio está INACTIVO o DETENIDO
Ejemplo: `Main PID: 1280 (code=exited, status=0/SUCCESS)`

- `1280` **(Último PID):** El número que tenía el proceso antes de cerrar.
- `code=exited` **(Razón de salida):** Indica la forma en que cerró. `exited` significa que el programa completó su orden de salida y se retiró (diferente a `killed`, que indicaría un cierre forzado o colapso por falta de memoria).
- `status=0` **(Código numérico de salida):** En los sistemas UNIX y Linux, el código **0** es el estándar universal para *"operación exitosa sin errores"*.
- `SUCCESS` **(Verificación):** La confirmación legible para el usuario de que el cierre fue limpio y planificado.

PREGUNTAS:

🗨 **1. Columna TTY y columna TIME**

- `?` **(En TTY):** Indica un proceso de fondo que no tiene una terminal o pantalla asignada.
- `tty1` **y** `tty2`**:** Son consolas virtuales (*Teletypes*). `tty1` corresponde a tu primer inicio de sesión en la máquina y `tty2` es la sesión activa en la que estás trabajando actualmente.
- `TIME` **en** `00:00:00`**:** Muestra el **tiempo acumulado de uso del procesador (CPU)**, no la hora del reloj. Como estos comandos consumen

solo milisegundos de procesamiento, el contador no llega a marcar un segundo.

2. Definición de `systemd` y `sd-pam`

- `systemd`**:** Es el proceso gestor maestro (PID 1) de Linux; se encarga de ***encender el sistema operativo*** y administrar todos los servicios.
- `sd-pam`**:** Es el módulo de ***autenticación*** (*Pluggable Authentication Module*) de `systemd` que valida las credenciales y permisos de tu usuario al ***iniciar sesión***.

3. Desglose del estado de `ssh.service`

- `Invocation`**:** ***Identificador*** único hexadecimal generado por el sistema en cada arranque del ***servicio*** para rastrear sus ***eventos*** en el ***registro global***.
- `TriggeredBy: ssh.socket`**:** Señala que un conector de red (*socket*) escucha el puerto 22 y activa o deriva las conexiones entrantes hacia el servicio SSH.
- `Docs` **(**`man:sshd(8)` / `man:sshd_config(5)`**):** Accesos a los manuales del sistema. El **(8)** indica la sección de *Comandos de Administración de Sistema* y el **(5)** la sección de *Formatos de Archivos de Configuración*.
- `Process`**:** ***Ejecución de prueba previa*** (`ExecStartPre`) que evalúa la configuración antes de levantar el servidor. El resultado `status=0/SUCCESS` confirma que la configuración no tiene errores.
- `Tasks: 1 (limit: 1882)`**:** El servicio tiene **1 proceso activo** corriendo en RAM, con un límite máximo de 1882 procesos permitidos para prevenir colapsos por sobrecarga.
- `Memory: 1.6M (peak: 2.4M)`**:** Indica que el servicio usa actualmente 1.6 Megabytes de memoria RAM, habiendo alcanzado un consumo máximo histórico (*peak*) de 2.4 Megabytes desde que inició.
- `CGroup` **(*Control Group*):** Es la estructura jerárquica con la que Linux limita y monitorea el uso de hardware para un grupo de procesos.
  - **Respuesta completa de CGroup:** Muestra que dentro del contenedor `/system.slice/ssh.service` se encuentra ejecutándose el proceso hijo con PID 1280 (`└─1280 "sshd: /usr/sbin/sshd -D..."`), el cual es el programa principal encargado de recibir las conexiones.

Preguntas CGroup y Docs:



La sección **Docs** contiene los enlaces al libro de instrucciones del servicio, mientras que **CGroup** es la caja donde Linux organiza el programa para limitar cuánto procesador y memoria puede consumir.

1. Docs (Los manuales del sistema)

El término `man` es la abreviatura de **manual**. En Linux, si quieres aprender a usar cualquier herramienta, consultas su página de manual.

El manual general de Linux está dividido en **8 capítulos temáticos** (como tomos de una enciclopedia) para no mezclar conceptos:

- **Tomo/Capítulo 5:** Reservado para **archivos de configuración** (explica cómo escribir las reglas dentro de un archivo).
- **Tomo/Capítulo 8:** Reservado para **comandos de administración del sistema** (herramientas avanzadas del administrador).

Desglosando tu pantalla:

- `man:sshd(8)` **:** Significa *"Si quieres leer cómo funciona el programa servidor SSH, abre el Capítulo 8 del manual"*.
- `man:sshd_config(5)` **:** Significa *"Si quieres aprender a editar el archivo de texto donde se configura SSH, abre el Capítulo 5 del manual"*.

2. CGroup (Control Group / Grupos de Control)

Imagina que Linux es una empresa y **CGroup** es el departamento de administración que le asigna a cada equipo una oficina fija con un límite de luz, agua y escritorios. Esto evita que un solo programa consuma todos los recursos de la máquina y la congele.

Desglosando tu pantalla:

Plaintext

```
CGroup: /system.slice/ssh.service
        └─1280 "sshd: /usr/sbin/sshd -D..."
```

- `/system.slice/ssh.service`: Es la "oficina" o carpeta de control que Linux le reservó a SSH.
- `└─1280`: Es el número de trabajador (PID 1280) que está trabajando dentro de esa oficina.
- `"sshd: /usr/sbin/sshd..."`: Es la tarea exacta que ese trabajador está realizando en este momento (mantener vivo el servidor SSH).

1. **Provocar Falla 1 (Servicio detenido):**

   `sudo systemctl stop ssh`

   `systemctl status ssh`

```
rafaelacordova@tel223-servidor2:~/laboratorio1$ sudo systemctl stop ssh
[sudo: authenticate] Password:
Stopping 'ssh.service', but its triggering units are still active:
ssh.socket
rafaelacordova@tel223-servidor2:~/laboratorio1$ systemctl status ssh
* ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
     Active: inactive (dead) since Tue 2026-08-25 18:49:07 UTC; 16s ago
   Duration: 1h 51min 31.005s
 Invocation: 5c32c7282b3e44f5b5a308dabe529500
TriggeredBy: • ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1274 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
    Process: 1280 ExecStart=/usr/sbin/sshd -D $SSHD_OPTS (code=exited, status=0/SUCCESS)
   Main PID: 1280 (code=exited, status=0/SUCCESS)
   Mem peak: 2.4M
        CPU: 73ms

ago 25 16:57:36 tel223-servidor2 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
ago 25 16:57:36 tel223-servidor2 sshd[1280]: Server listening on 0.0.0.0 port 22.
ago 25 16:57:36 tel223-servidor2 sshd[1280]: Server listening on :: port 22.
ago 25 16:57:36 tel223-servidor2 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
ago 25 18:49:07 tel223-servidor2 sshd[1280]: Received signal 15; terminating.
ago 25 18:49:07 tel223-servidor2 systemd[1]: Stopping ssh.service - OpenBSD Secure Shell server...
ago 25 18:49:07 tel223-servidor2 systemd[1]: ssh.service: Deactivated successfully.
ago 25 18:49:07 tel223-servidor2 systemd[1]: Stopped ssh.service - OpenBSD Secure Shell server.
rafaelacordova@tel223-servidor2:~/laboratorio1$ _
```

🗨 El resultado es **100 % correcto**: ejecutaste con éxito la detención del servicio SSH para simular la **Falla 1** descrita en la guía y confirmaste que el sistema registró el evento adecuadamente.
Explicación teórica del comando (`sudo systemctl stop ssh`)

- `systemctl stop`: Envía una orden directa al gestor del sistema (`systemd`) para detener la ejecución de un servicio activo.

- **Aviso** `Stopping 'ssh.service', but its triggering units are still active: ssh.socket` : En versiones recientes de Ubuntu, la red escucha las peticiones entrantes mediante un conector (*socket*). Este mensaje te advierte que el servicio principal se apagó, pero el conector del puerto sigue alerta en caso de recibir una nueva solicitud.

Análisis de los resultados en la pantalla (`systemctl status ssh`)

- `Active: inactive (dead)` : Es la prueba principal de la falla. El estado cambió de verde (`running`) a un texto gris que indica **inactivo/muerto**.
- `Main PID: 1280 (code=exited, status=0/SUCCESS)` : Confirma que el proceso número `1280` que estaba guardado en la memoria RAM se cerró. El código `status=0` significa que finalizó limpiamente y sin colisionar.
- `Received signal 15; terminating` : Los registros del final indican que Linux le envió la **Señal 15 (SIGTERM)** al programa. Es una orden estándar que le pide al proceso cerrar sus operaciones de forma ordenada.
- `ssh.service: Deactivated successfully` : Registro definitivo del sistema operativo validando que el servicio SSH fue desactivado con éxito.

Siguientes pa

`journalctl -u ssh -n 20`

> 📖 **El desglose paso a paso**
> • `journalctl` *(El cuaderno del guardia)*: Es la orden de abrir el libro donde se registran todas las actividades del sistema. Le estás diciendo a Linux: *"Oye, quiero ver el historial de cosas que han pasado aquí"*.
> • `-u ssh` *(El departamento específico)*: La `-u` significa "Unidad" (un programa o servicio). En este caso, le especificas que solo te interesan las notas de `ssh`. El `ssh` es como la **puerta trasera del edificio** por donde la gente entra a trabajar de forma remota desde sus casas. Así que aquí dices: *"Solo muéstrame el historial de la puerta trasera"*.
> • `-n 20` *(Las últimas notas)*: La `-n` significa "Número". Como el cuaderno es

> gigante y tiene miles de páginas, el `20` le dice al guardia: *"No me des todo el libro, solo muéstrame las **últimas 20 anotaciones** que hiciste"*.

```
rafaelacordova@tel223-servidor2:~/laboratorio1$ journalctl -u ssh -n 20
ago 25 16:57:36 tel223-servidor2 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
ago 25 16:57:36 tel223-servidor2 sshd[1280]: Server listening on 0.0.0.0 port 22.
ago 25 16:57:36 tel223-servidor2 sshd[1280]: Server listening on :: port 22.
ago 25 16:57:36 tel223-servidor2 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
ago 25 18:49:07 tel223-servidor2 sshd[1280]: Received signal 15; terminating.
ago 25 18:49:07 tel223-servidor2 systemd[1]: Stopping ssh.service - OpenBSD Secure Shell server...
ago 25 18:49:07 tel223-servidor2 systemd[1]: ssh.service: Deactivated successfully.
ago 25 18:49:07 tel223-servidor2 systemd[1]: Stopped ssh.service - OpenBSD Secure Shell server.
rafaelacordova@tel223-servidor2:~/laboratorio1$
```

`sudo systemctl start ssh`

`systemctl status ssh`

```
rafaelacordova@tel223-servidor2:~/laboratorio1$ sudo systemctl start ssh
rafaelacordova@tel223-servidor2:~/laboratorio1$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
     Active: active (running) since Tue 2026-08-25 18:57:54 UTC; 17s ago
 Invocation: 716a6134997e420b9156300afdddcb13
TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 2165 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 2168 (sshd)
      Tasks: 1 (limit: 1882)
     Memory: 1.4M (peak: 2M)
        CPU: 40ms
     CGroup: /system.slice/ssh.service
             └─2168 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

ago 25 18:57:54 tel223-servidor2 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
ago 25 18:57:54 tel223-servidor2 sshd[2168]: Server listening on 0.0.0.0 port 22.
ago 25 18:57:54 tel223-servidor2 sshd[2168]: Server listening on :: port 22.
ago 25 18:57:54 tel223-servidor2 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
rafaelacordova@tel223-servidor2:~/laboratorio1$
```

> En administración de sistemas, apagar un servicio deliberadamente mediante una **Señal 15 (`SIGTERM`)** se utiliza como una **falla simulada**. Para un usuario externo que intenta conectarse por SSH, el servidor no responde y la aplicación está "caída"; el motivo exacto de esa indisponibilidad es precisamente lo que debes aprender a investigar.
>
> **Por qué la Señal 15 es clave para el diagnóstico**
>
> - **Simulación de apagado controlado:** La Señal 15 (`SIGTERM`) es una solicitud de terminación limpia que le da tiempo al proceso para guardar datos y cerrarse. En el mundo real, esta situación ocurre si un administrador apaga el servicio por error, si un script automatizado lo detiene sin avisar, o si un proceso de actualización lo deshabilita.

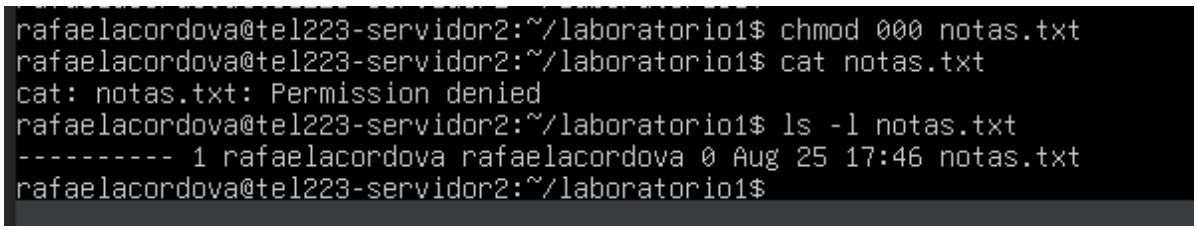
> - **Diferenciación de causa raíz:** Diagnosticar consiste en determinar las circunstancias del fallo. Si el registro muestra `code=exited, status=0/SUCCESS` y `Received signal 15`, tu investigación concluye que **el programa no colapsó por un error de programación o falta de memoria**, sino que **recibió una orden explícita de apagado**.
>
> **Aplicación práctica de la Regla de Diagnóstico**
>
> Para cumplir con la competencia de tu laboratorio, debes estructurar la investigación siguiendo el método científico (Hipótesis $\rightarrow$ Predicción $\rightarrow$ Comprobación):
>
> - **1. Hipótesis:** "Si ejecuto `sudo systemctl stop ssh`, la herramienta `systemd` enviará una orden de apagado ordenado (`SIGTERM`) al proceso principal (PID 1280)".
> - **2. Lo que esperas ver (Predicción):**
>   - En `systemctl status`: El indicador de estado cambiará de `active (running)` en verde a `inactive (dead)` en gris.
>   - En el historial (`journalctl`): Aparecerán las líneas `Received signal 15` y `Deactivated successfully`.
> - **3. Comprobación:** Ejecutas el comando de inspección y validas la pantalla. Si el resultado coincide con lo previsto, has verificado tu hipótesis y diagnosticado con éxito que el servicio simplemente requiere un reinicio (`start`).
>
> Esta dinámica te permite distinguir visualmente entre un servicio **detenido administrativamente** (Señal 15) y un servicio **colapsado por falla crítica** (Señal 9 / *Crash*).

1. **Provocar Falla 2 (Permisos eliminados):**

   `chmod 000 notas.txt`

   `cat notas.txt`

   `ls -l notas.txt`



El resultado es **100 % correcto** y demuestra cómo Linux aplica sus reglas de seguridad sin excepciones: al quitarle todos los permisos a un archivo, ni siquiera su propio dueño puede leerlo.

Explicación teórica (`chmod 000`)

El código `000` asigna 0 puntos de acceso a las tres capas de seguridad:

• **Primer `0` (Tú / Propietario):** Te retira tus propios derechos de lectura, escritura y ejecución.

• **Segundo `0` (Tu grupo):** Le quita cualquier tipo de acceso a los miembros de tu grupo.

• **Tercer `0` (Otros usuarios):** Bloquea completamente a cualquier otra persona en el servidor.**Regla fundamental de Linux:** Las reglas de permisos se aplican sin importar quién seas. Aunque tú seas el dueño registrado (`rafaelacordova`), si la etiqueta del archivo dice que nadie tiene permiso de lectura, el sistema operativo te bloqueará la puerta.

Análisis de los resultados en la imagen

1. `cat notas.txt` **$\rightarrow$** `cat: notas.txt: Permission denied`
   ◦ El comando `cat` se utiliza para abrir y leer el contenido de un archivo.
   ◦ Como le quitaste el permiso de lectura (`r`), el sistema intercepta la orden y muestra `Permission denied` (*Permiso denegado*).
2. `ls -l notas.txt` **$\rightarrow$** `---------- 1 rafaelacordova rafaelacordova ...`
   ◦ **El primer guion `-`:** Confirma que es un archivo normal.
   ◦ **Los 9 guiones seguidos (`---------`):** Es la confirmación visual absoluta de que la tabla de permisos está completamente vacía (sin `r`, sin `w` y sin `x` para nadie).

```
chmod 644 notas.txt
```

```
cat notas.txt
```

Falla 1: el servicio detenido

Regla del diagnóstico: antes de cada comando, escriba qué espera ver. Diagnosticar no es probar cosas al azar: es formular una hipótesis y comprobarla.

27. Detenga el servicio y observe:

```
sudo systemctl stop ssh
systemctl status ssh
journalctl -u ssh -n 20
```

Qué cambió en el estado	Qué dice el registro	Causa
Active: inactive (dead)	ssh.service:Deactivated succesfully	El comando sudo systemctl stop ssh

28. Vuelva a activarlo y confirme que quedó activo:

```
sudo systemctl start ssh
systemctl status ssh
```

Falla 2: el permiso quitado

29. Quite todos los permisos a su archivo e intente leerlo:

```
chmod 000 notas.txt
cat notas.txt
ls -l notas.txt
```

Qué mensaje apareció	Qué muestra ls -l	Causa
cat: notas.txt: Permission denied	——— - del documento normal ——— de los permisos quitados	Quitar los permisos con los dígitos en cero 000

30. Devuelva los permisos y compruebe que puede leerlo otra vez:

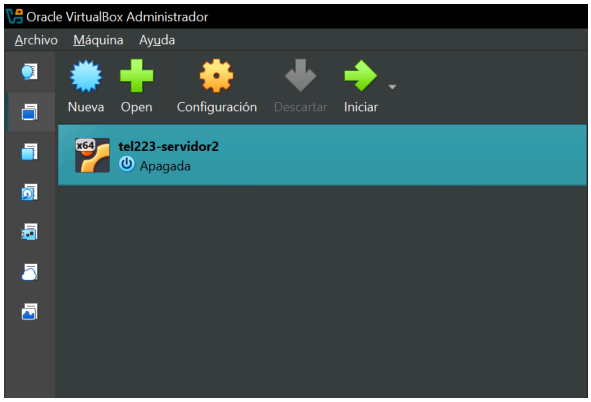
```
chmod 644 notas.txt
cat notas.txt
```

La idea que hay que llevarse: un permiso mal puesto es la causa más frecuente de que un servicio no arranque. No siempre que algo falla está roto: a veces solo no tiene permiso.

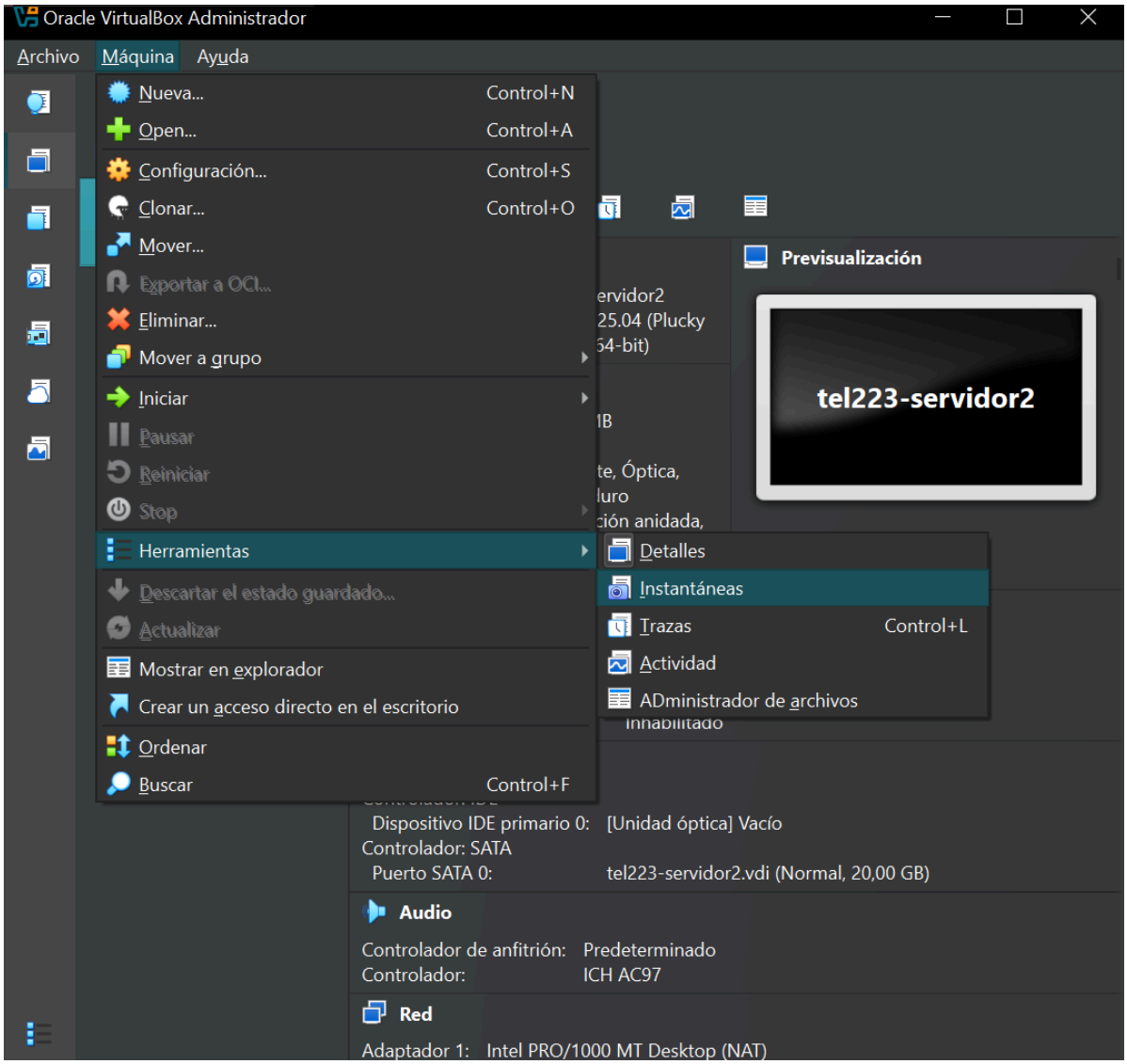
ETAPA 7: Apagado e Instantánea

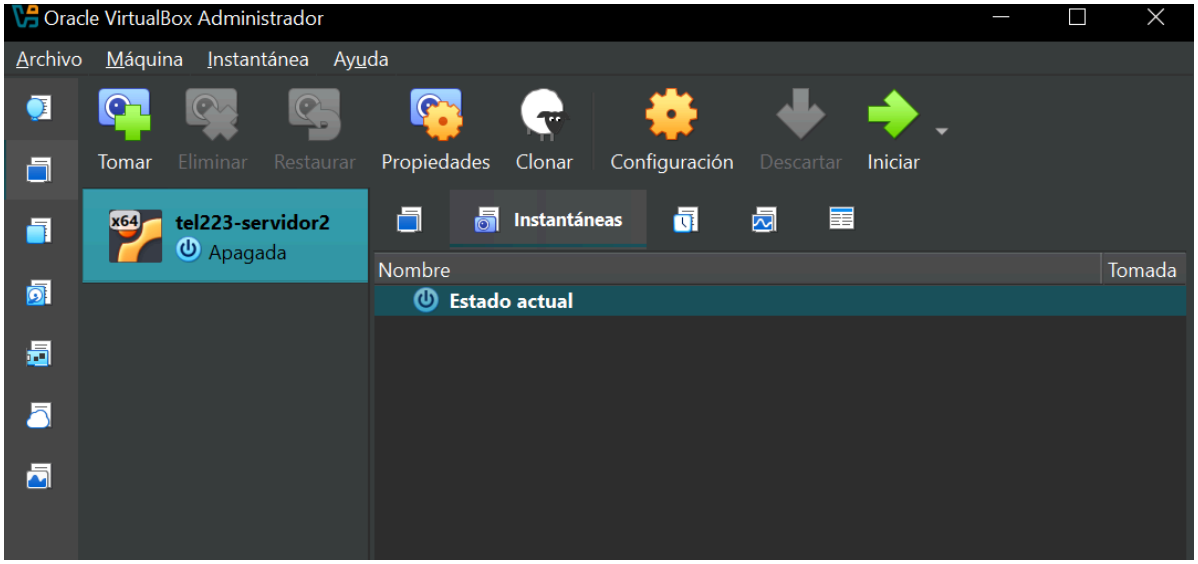
1. Apaga el servidor de forma segura desde la consola:
   `sudo poweroff`

[después de haber colocado la *contraseña* en consola...]

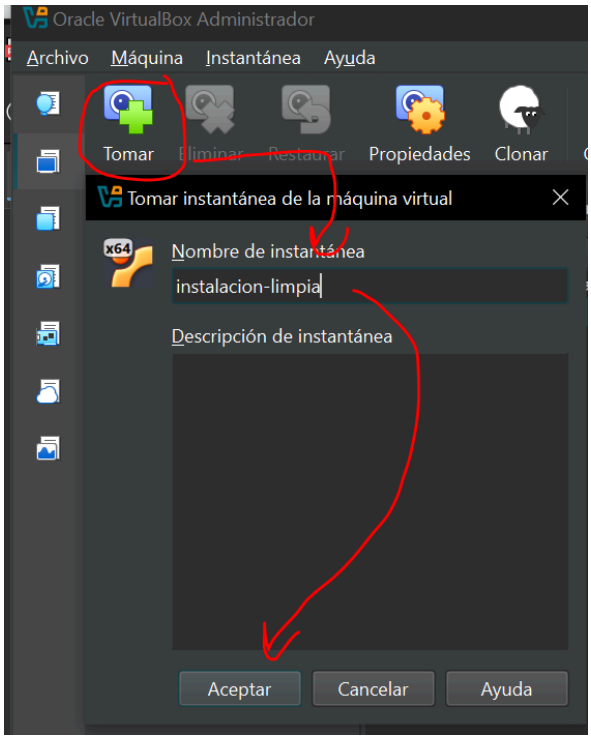


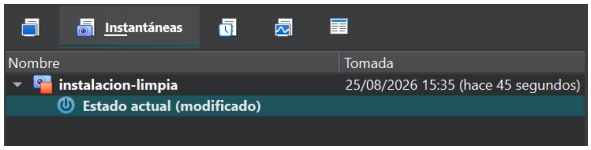
1. Una vez que la VM esté apagada, dirígete a VirtualBox:
   - Selecciona la máquina `tel223-servidor2` -> Menú **Máquina** -> Opción **Instantáneas** (Snapshots).


Oracle VirtualBox Administrador
Archivo
Máquina
Ayuda
Nueva... Control+N
Open... Control+A
Configuración... Control+S
Clonar... Control+O
Mover...
Exportar a OCI...
Eliminar...
Mover a grupo
Iniciar
Pausar
Reiniciar
Stop
Herramientas
Descartar el estado guardado...
Actualizar
Mostrar en explorador
Crear un acceso directo en el escritorio
Ordenar
Buscar Control+F
Detalles
Instantáneas
Trazas Control+L
Actividad
ADministrador de archivos
Previsualización
tel223-servidor2
Dispositivo IDE primario 0: [Unidad óptica] Vacío
Controlador: SATA
Puerto SATA 0: tel223-servidor2.vdi (Normal, 20,00 GB)
Audio
Controlador de anfitrión: Predeterminado
Controlador: ICH AC97
Red
Adaptador 1: Intel PRO/1000 MT Desktop (NAT)


Oracle VirtualBox Administrador
Archivo
Máquina
Instantánea
Ayuda
Tomar
Eliminar
Restaurar
Propiedades
Clonar
Configuración
Descartar
Iniciar
tel223-servidor2
Apagada
Instantáneas
Nombre
Tomada
Estado actual

- Haz clic en **Tomar** y ponle de nombre: `instalacion-limpia`





n